Лабораторная работа № 3. Шаблоны. Админка Django

Выведем данные, полученные в консоле в браузере. Для этого запустим сервер и откроем главную страницу с “Hello world”.

Необходимо получить данные из модели. Для этого перейдем в папку views.py и импортируем модель (рисунок 1).

```
from .models import News
```

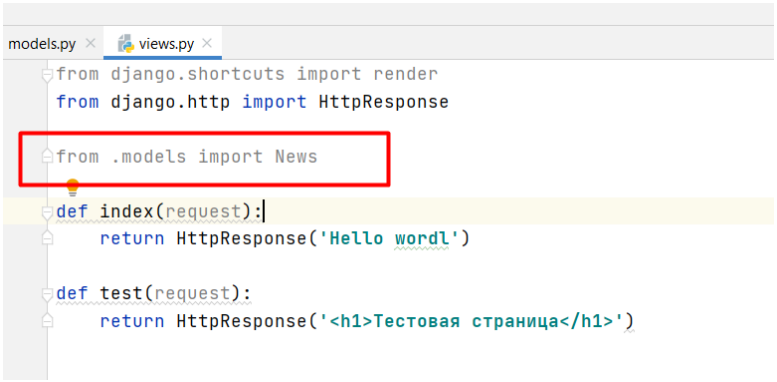


Рисунок 1

Теперь мы готовы получить эти данные через создаваемую переменную news и соответственно обратимся к модели News менеджеру объектов и методом all() получим все новости (рисунок 2).

```
news = News.objects.all()
```



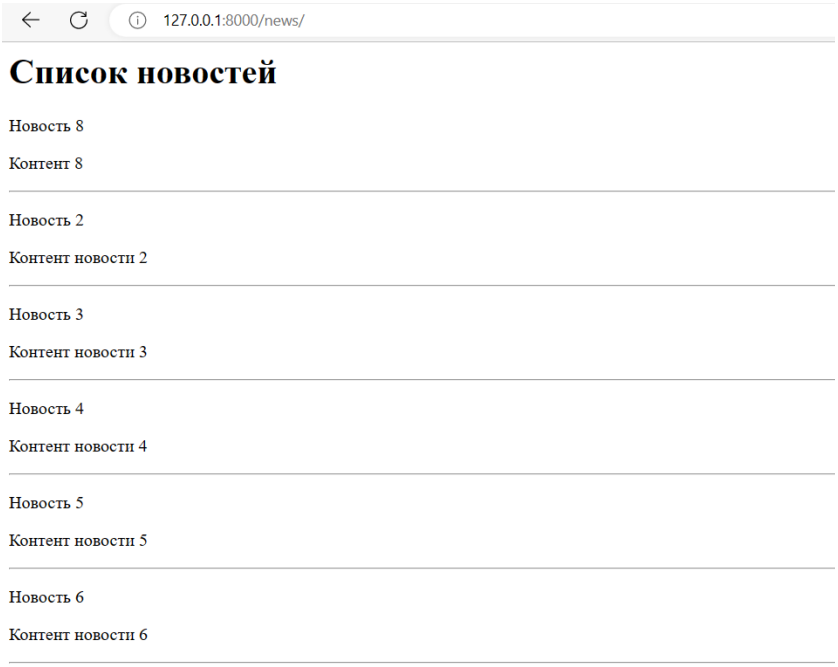
Рисунок 2

Теперь сформируем в следующей строке контроллера index ответ, который будем показывать пользователю (рисунок 3)

```python
def index(request):
    news = News.objects.all()
    res = '<h1>Список нововстей</h1>'
    for item in news:
        res+=f'<div>\n<p>{item.title}</p>\n<p>{item.content}</p>\n</div>\n<hr>'
    return HttpResponse(res)
```

Рисунок 3

Обновляем страницу (рисунок 4)



Можно посмотреть исходный код (рисунок 5) (правая кнопка мыши выбрать в меню Просмотр исходного кода (рисунок 4).

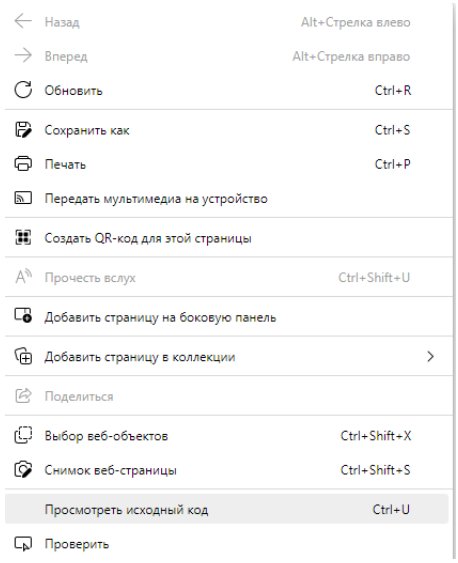


Рисунок 4

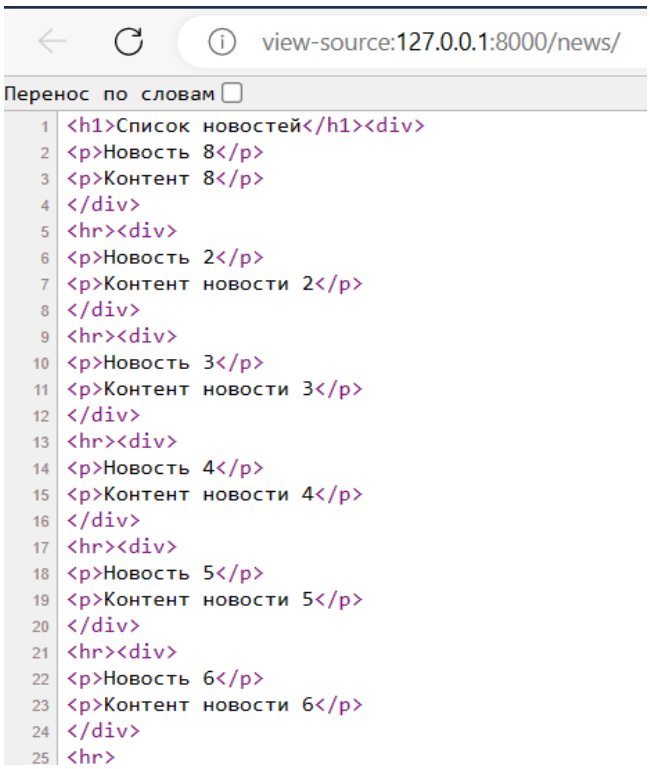


Рисунок 5

Видно, что здесь не хватает стандартных тегов <heat> <html>, <body> и др.

Конечно, мы это можем дописать все эти основные теги в строках цикла, но тогда получился бы очень большой код. А еще если подключать стили, скрипты, то это еще больше увеличило код этого контроллера.

Чтобы этого избежать, используются для этого шаблоны. Шаблоны, которые мы можем рендерить.

Рендеринг – процесс, наполнения файла данными из модели, т.е. формирование страницы с данными. Django позволяет рендерить с шаблона и работает с помощью специального шаблонизатора.

Это можно посмотреть в документации Django (https://django.fun/ru/) (рисунок 6)

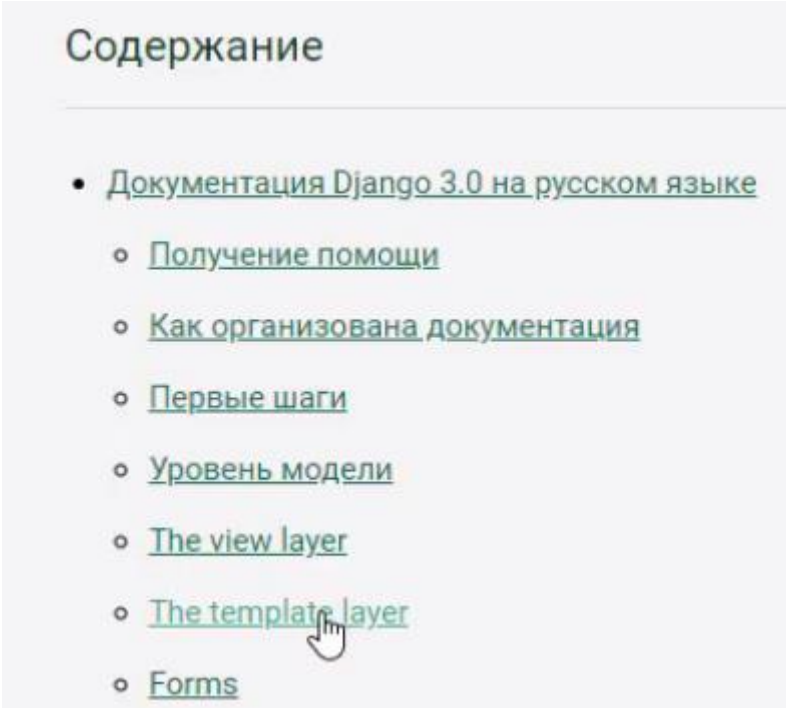


Рисунок 6

Шаблонов много с их описанием можно познакомится например, на русско-язычном сайте (рисунок 7).

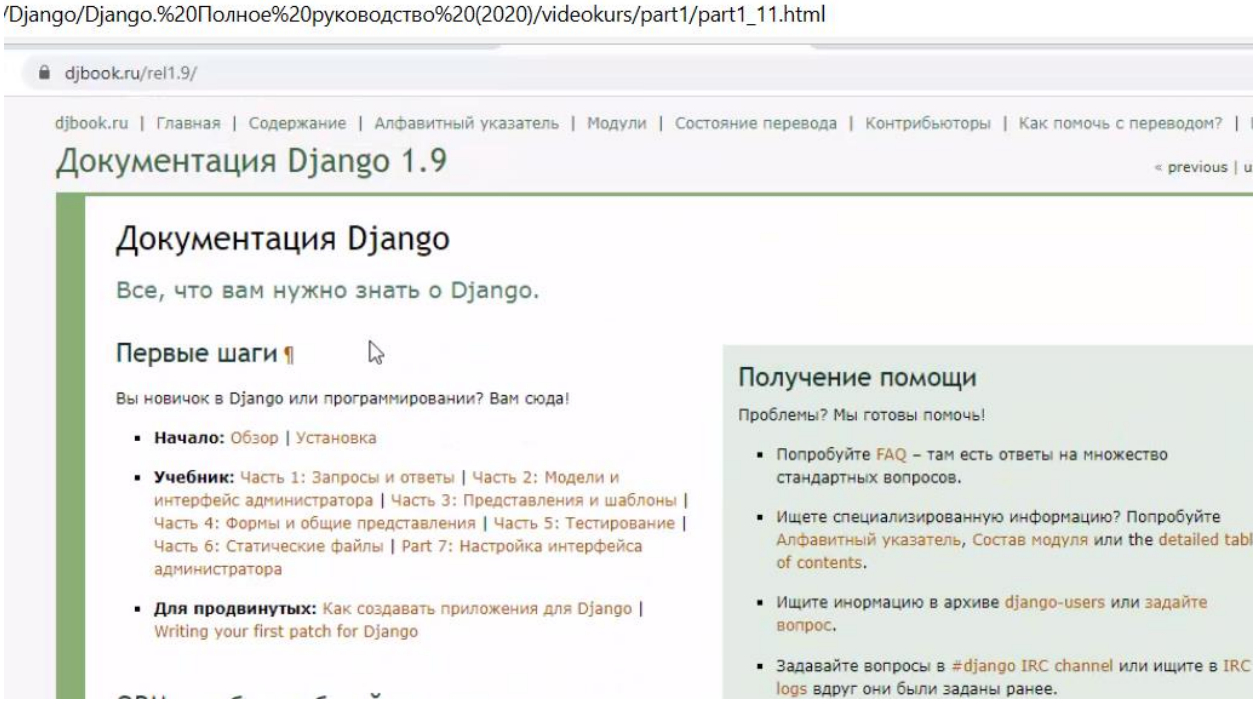


Рисунок 7

Когда мы работаем с шаблонизатором, то мы по сути работаем в нем с тремя объектами: директивы, теги, фильтры.

Рассмотрим пока некоторые базовые вещи.

По умолчанию Django ищет шаблоны в папке templates в соответствующем приложении. По умолчанию эта папка не создаются. Поэтому ее необходимо создать (рисунок 8, 9).

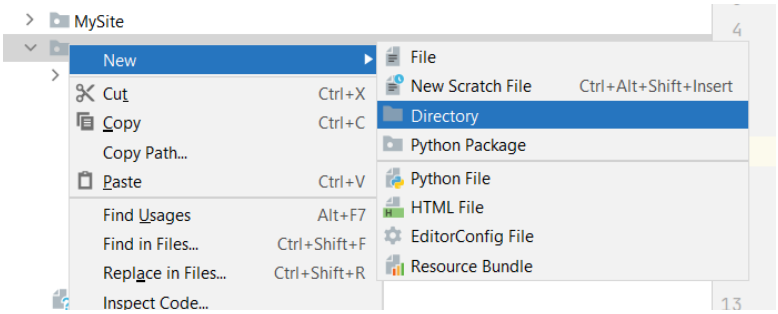


Рисунок 8

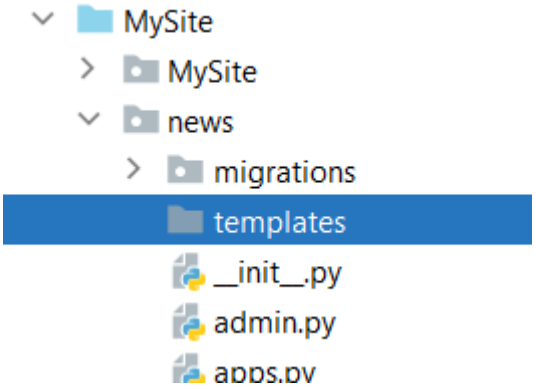


Рисунок 9

Django очень гибко подходит к работе с шаблонами и их можно помещать куда угодно, это все настраивается и Django будет искать там где будет указано. Но Django рекомендует внутри папки templates создать еще одноименную с приложением папку, в нашем случае, news и туда складывать шаблоны (рисунок 10).

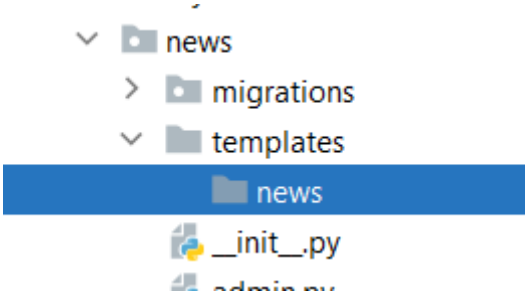


Рисунок 10

Почему рекомендуется именно так? На этапе компиляции, т.е. сбора приложения, Django собирает все шаблоны в одну кучу (локализует) в одном

месте и будет указывать первый шаблон, указанный для рендеринга. Но если, например, создадим контроллер index в другом приложении, подобный индексу в приложении news, то возникнет проблема конфликта, и чтобы этого не было, будем использовать некоторое пространство имен, которым является еще один подкаталог.

Шаблон – это обычный HTML файл. Создадим его и назовем index

Как правило, шаблон создается по названию action (т.е. по названию того контроллера, который возвращает нам ответ (рисунок 11).

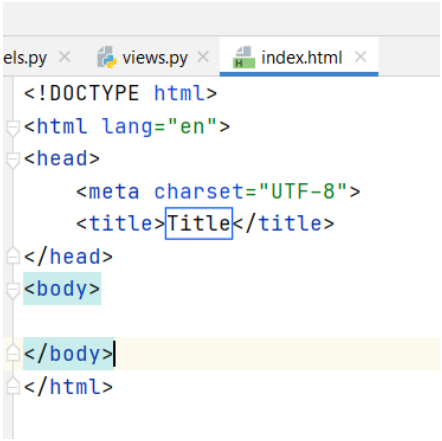


Рисунок 11

Отредактируем этот файл в соответствии с рисунком

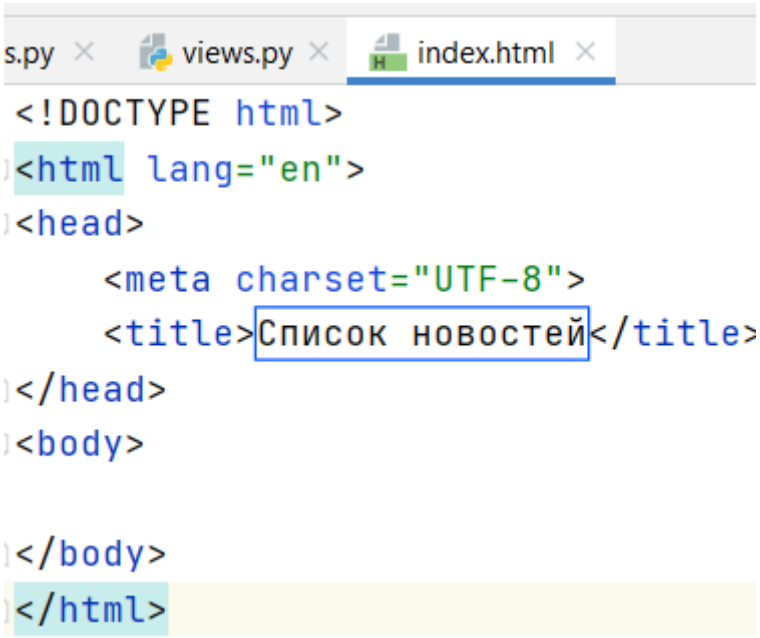


Рисунок 12

Для того, чтобы подключить шаблон, т.е. провести рендеринг, мы должны использовать специальную функцию из модуля shortcuts (рисунок 13)

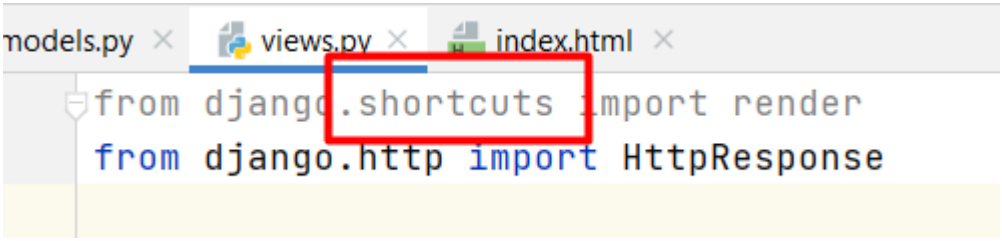


Рисунок 13

Из этого модуля импортируется функция render.

Изменим контроллер index вплоть до значения return, который будет возвращать render.

Render возвращает следующие параметры: параметр request наш объект запроса, название шаблона, который будет использоваться при загрузке данных и также принимает контекст.

Контекст – это словарь, с данными, которые необходимо передать в шаблон, в нашем случае news. Ключ в словаре можно называть любым именем, но для прозрачности кода, лучше задать его одноименным (рисунок 14).

```
def index(request):
    news = News.objects.all()
    return render(request, 'news/index.html',{'news': news, 'title': 'Список новостей'})
```

Рисунок 14

Снова перейдем в наш шаблон.

Как говорилось, шаблонизатор Django предлагает три вещи: директивы, теги, фильтры.

Директивы – это по сути переменные, которые позволяют выводить значения, которые передали в шаблон. В нашем случае это

```
'news': news, 'title': 'Список новостей'
```

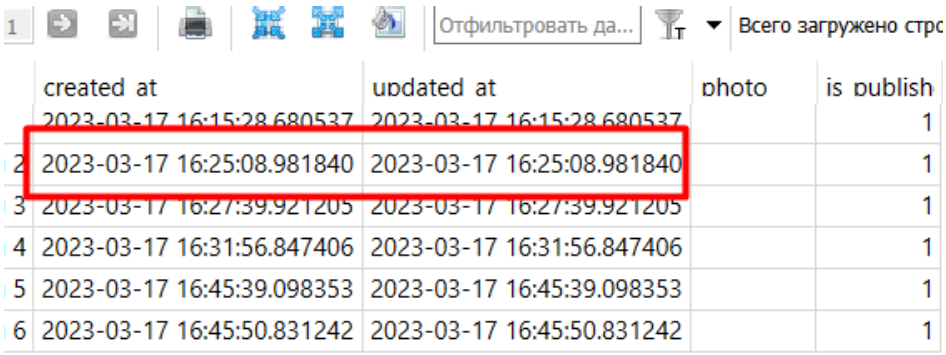
И здесь этими переменными являются 'news' и 'title'

В 'title' будет строка, в 'news' – список, который нужно итерировать. Для того, чтобы итерировать нам понадобятся не только директивы, но и теги.

Теги позволяют производить какие-то действия, они управляют генерированием вывода. Например, есть тег for, if и т.д. По сути, они дублируют синтез Python.

Фильтры – занимаются тем, что выполняют преобразованием перед выводом.

Например, если посмотрим на поле date, то увидите не очень понятный формат (рисунок 15)



	created at	updated at	photo	is publish
	2023-03-17 16:15:28.680537	2023-03-17 16:15:28.680537		1
2	2023-03-17 16:25:08.981840	2023-03-17 16:25:08.981840		1
3	2023-03-17 16:27:39.921205	2023-03-17 16:27:39.921205		1
4	2023-03-17 16:31:56.847406	2023-03-17 16:31:56.847406		1
5	2023-03-17 16:45:39.098353	2023-03-17 16:45:39.098353		1
6	2023-03-17 16:45:50.831242	2023-03-17 16:45:50.831242		1

Рисунок 15

Поэтому необходимо вывести его в более читаемом формате. И в этом случае используются фильтры.



Рисунок 16

И обновим страницу в браузере (рисунок 17).

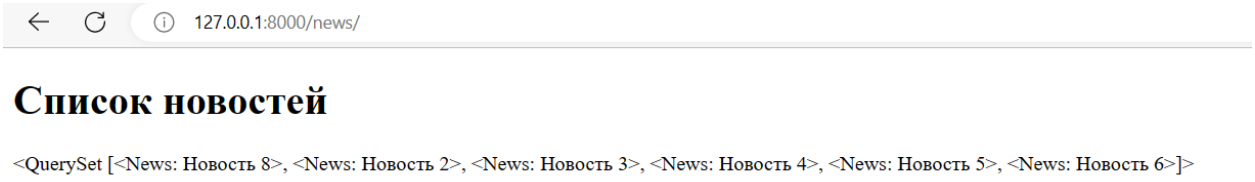


Рисунок 17

Как мы видим, что вывод не совсем в «читабельном» виде и это конечно, необходимо исправить. И это можно исправить с помощью тегов, которые управляют генерированием вывода. Например, используем тег for, который организует обычный цикл по объектам.

Для того чтобы его создать, используется следующая запись

```
{% block %}

{% endblock %}
```

Где под block понимается какой-либо тег, реализующий вывод. Например, тег for (рисунок 18).

```
<body>
<h1>{{title}}</h1>

{% for item in news %}
    <div>
        <p>{{item.title}}</p>
        <p>{{item.content}}</p>
        <p>{{item.created_at}}</p>
    </div>
<hr>
{% endfor %}

</body>
```

Рисунок 18

Обновим страницу браузера и увидим уже красивый формат вывода (рисунок 19)



Рисунок 19

Но мы видим, что дата представлена в англо-язычном формате. Для рядового пользователя такой формат времени может быть не понятным. Поэтому внесем поправки. Это можно сделать двумя способами.

Во-первых, можно обратиться к настройкам нашего приложения settings (рисунок 20).

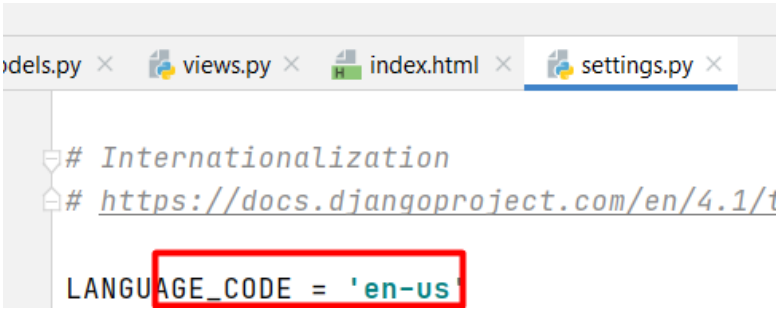


Рисунок 21

Это означает, что приложение использует этот язык и все настройки локализации отталкиваются от этого языка. Чтобы изменить на русский язык можно изменить на ru (рисунок 22).

```
# Internationalization
# https://docs.djangoproject.com/

LANGUAGE_CODE = 'ru'
```

Рисунок 23

После обновления видим список новостей в удобном для нас формате (рисунок 24)

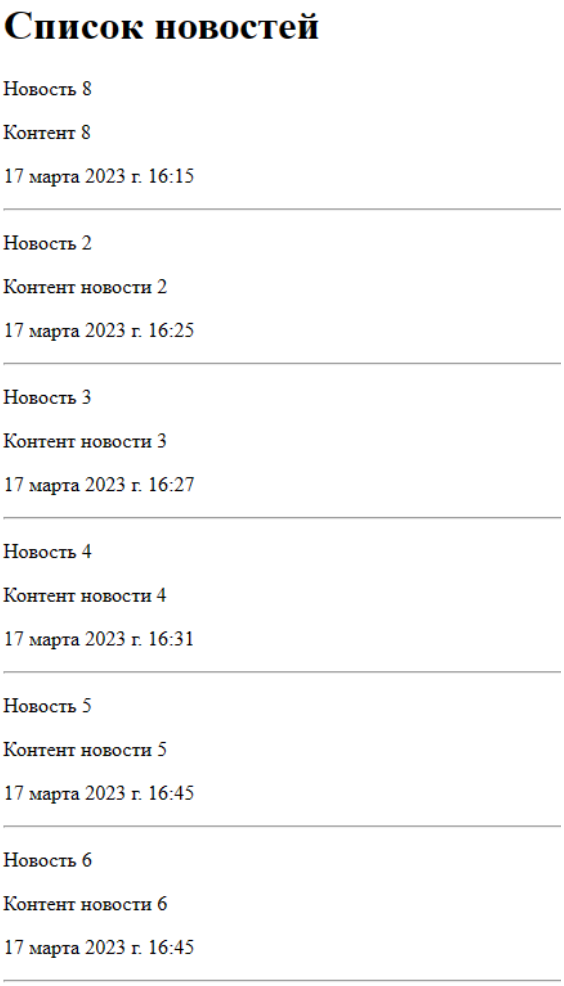


Рисунок 24

Рассмотрим другой вариант, более гибкий с помощью фильтров (рисунок 25).

```
    <p>{{item.created_at|date:"Y-m-d H:i:s"}}</p>
</div>
```

Рисунок 25

После обновления видим улучшенный вариант(рисунок 26)

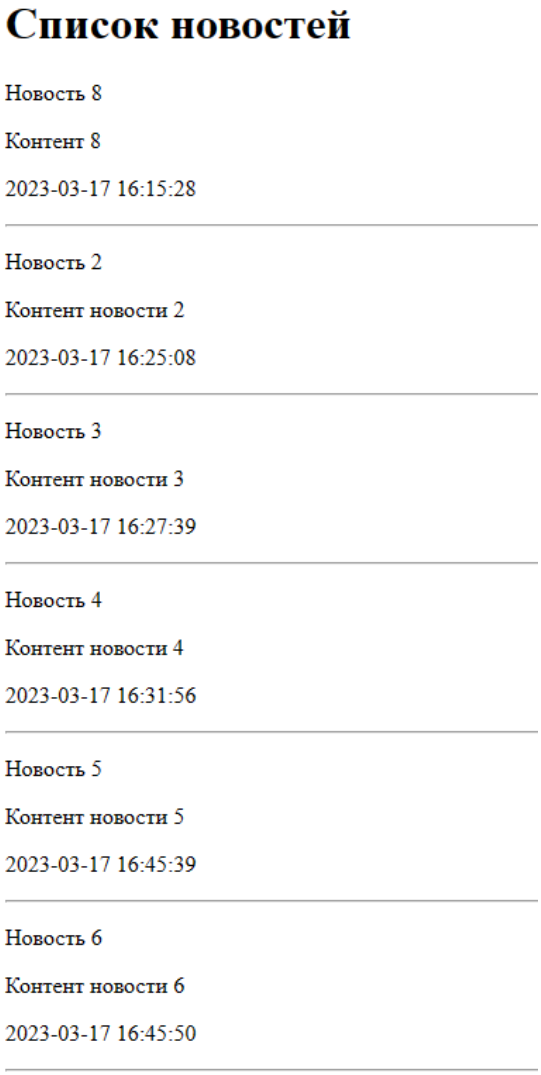


Рисунок 26

Создавая ответ строкой

```
(request, 'news/index.html',{'news': news, 'title': 'Список новостей'})
```

Третьим параметром, мы создавали контекст. Но если, контекста много, то в этом случае создают переменную, которая будет словарем содержать передаваемые значениям (рисунок 27).

```
def index(request):
    news = News.objects.all()
    context={
        'news': news,
        'title': 'Список новостей'
    }
    return render(request, 'news/index.html',context)
```

Рисунок 27

Здесь мы использовали позиционный документ, но можно использовать именованные аргументы (рисунок 28).

```
    'news': news,
    'title': 'Список новостей'
}
return render(request, template_name='news/index.html',context=context)
```

Рисунок 28

Таким образом, мы получили готовую HTML страницу с выводом данных модели.

Добавим еще одно действие, которое расположит новости по дате создания. Для этого в контроллере index заменим all() на order_by() и в нем указать, по какому полю мы хотим отсортировать в обратном порядке. Знак минус указывает на сортировку в обратном порядке `order_by('-created_at')` (рисунок 29).

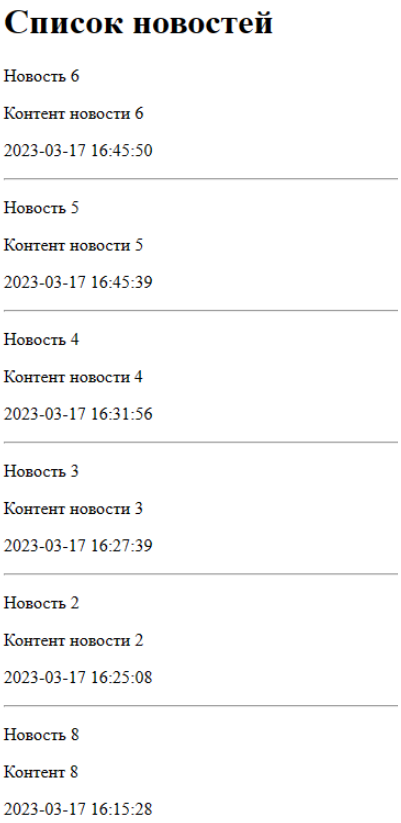


Рисунок 29

Перейдем теперь к работе с админкой.

Как известно, админка уже доступна по умолчанию.

Откроем стартовую страницу админки (рисунок 30) по адресу http://127.0.0.1:8000/admin

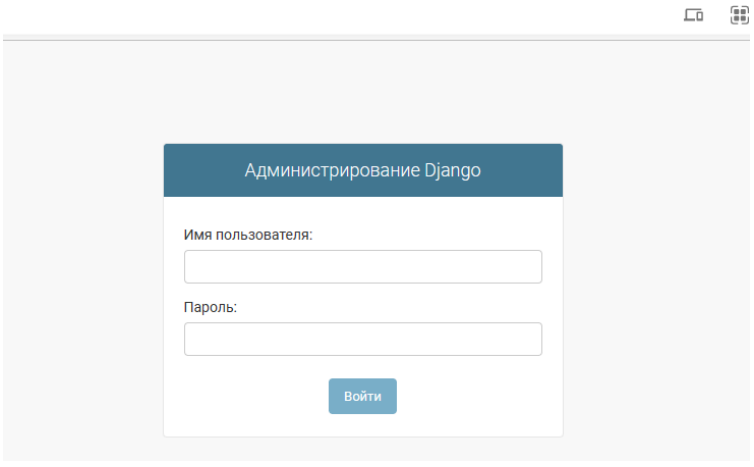


Рисунок 30

Как видно, необходимо авторизоваться, но для этого необходимо создать суперпользователя.

Для этого перейдем в консоль терминала и введем команду

python manage.py createsuperuser

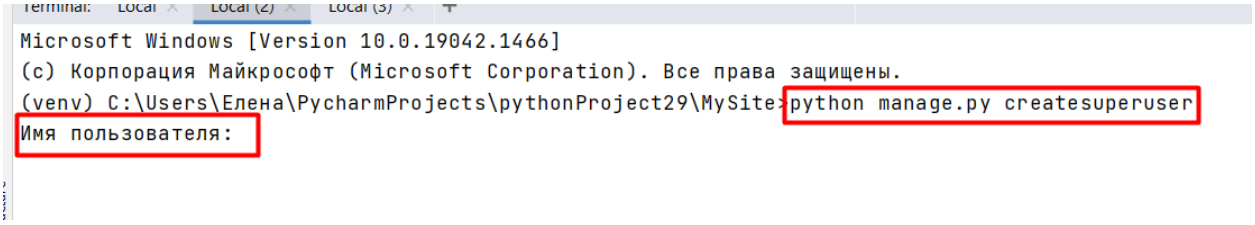


Рисунок 31

После ввода команды будет предложено ввести Имя пользователя, например, admin (совет, в реальных проектах использовать сложное имя). Назначаем имя суперпользователь нажимаем enter (рисунок 32).

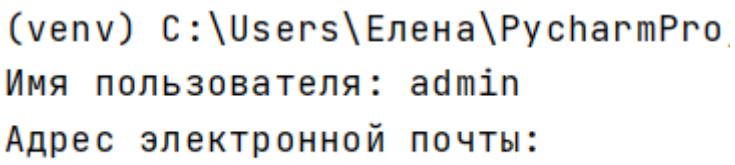


Рисунок 32

Далее будет предложено ввести адрес электронной почты, которое является необязательным. Введем для примера admin@localhost

Далее предлагается ввести пароль. Если вы введете короткий пароль, то Django сообщит вам об этом и даст совет о вводе более длинного пароля и предложит подтвердить уже введенный пароль введением Y или, если вы введете N, то получите возможность назначить другой более сложный пароль (рисунок 33).

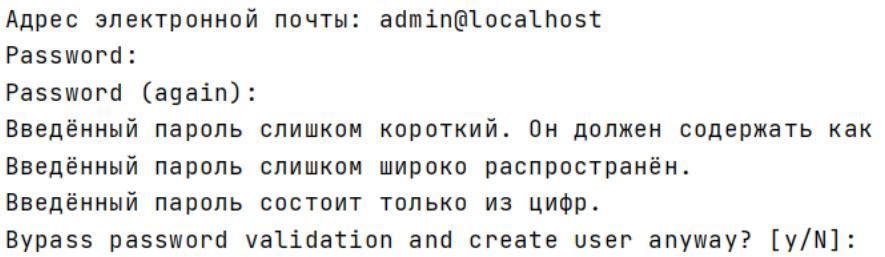


Рисунок 33

Обратите внимание, что при вводе пароля не отображаются символы ввода, для того чтобы посторонний пользователь не мог его подсмотреть.

После подтверждения пароля вы увидите подтверждение об успешном назначении этого пароля суперпользователю (рисунок 34).

```
Введённый пароль состоит только из цифр.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Рисунок 34

Теперь давайте зайдем под этими данными в админку (рисунок 35).

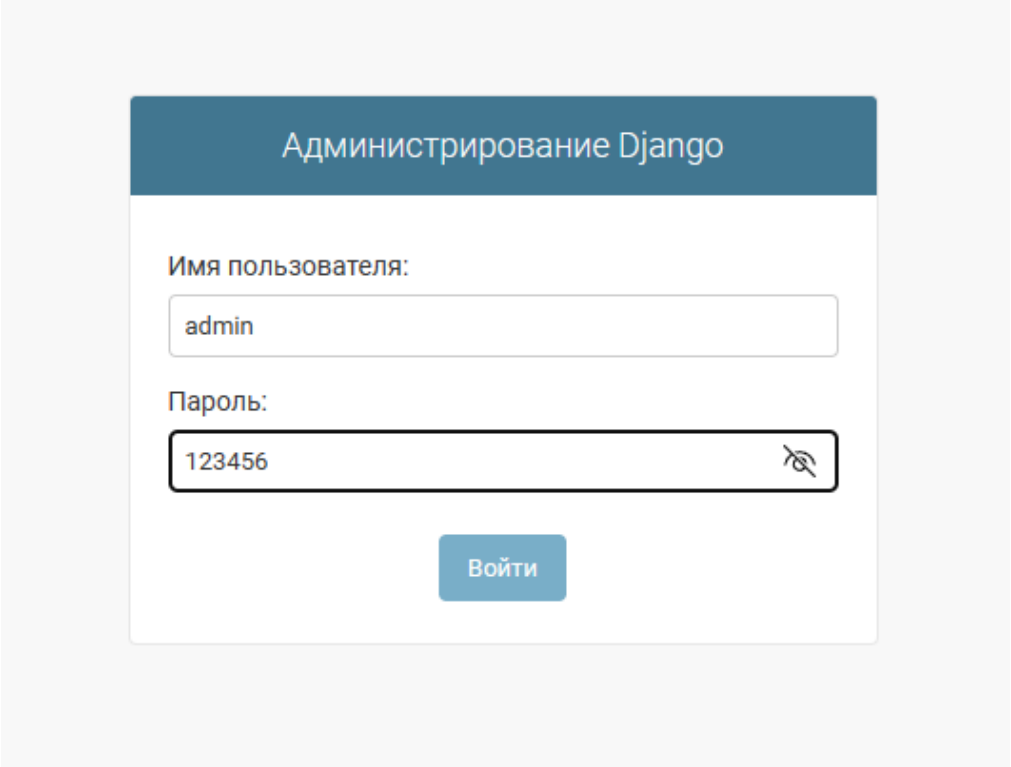


Рисунок 35

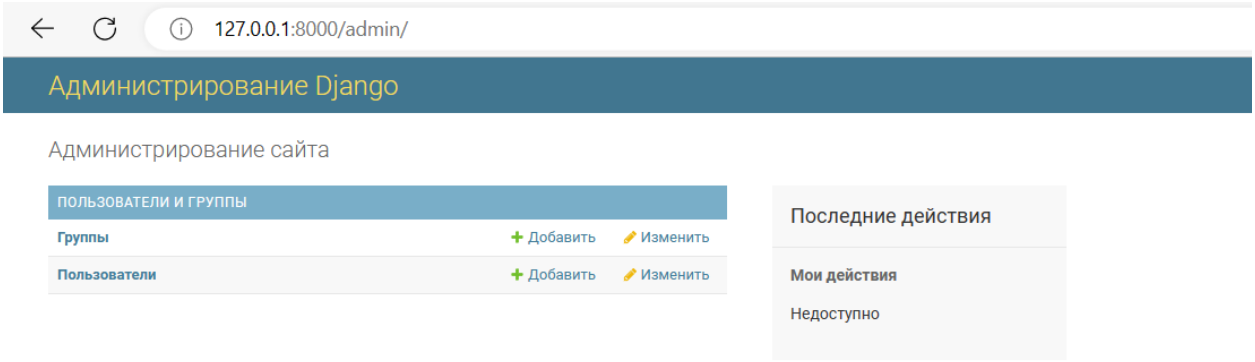


Рисунок 36

Как вы видите, админка здесь уже русифицирована. По умолчанию уже здесь зарегистрировано два приложения: Группа, Пользователи.

Нажав на знак + мы можем здесь добавлять пользователей, редактировать (рисунок 37)

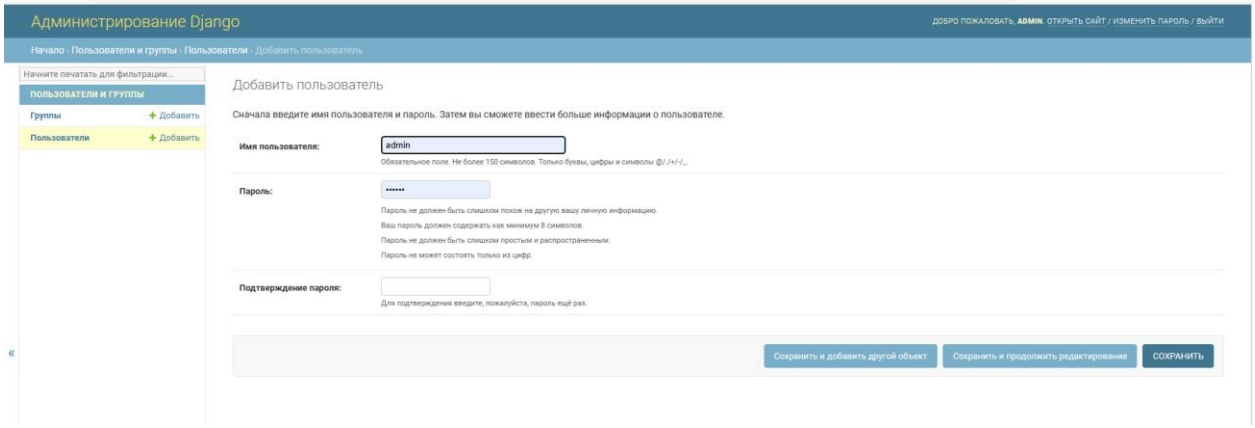


Рисунок 37

Можем добавлять группы (рисунок 38)



Рисунок 38

Назначать роли пользователям и группам (рисунок 39)

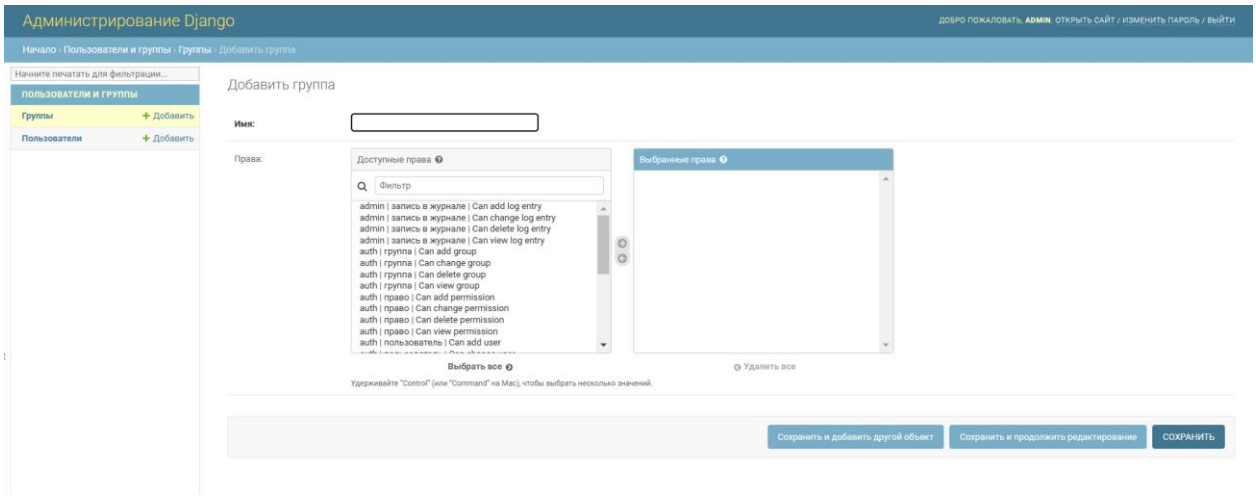


Рисунок 39

Справа в панели админки вы видите список действий (рисунок 40)

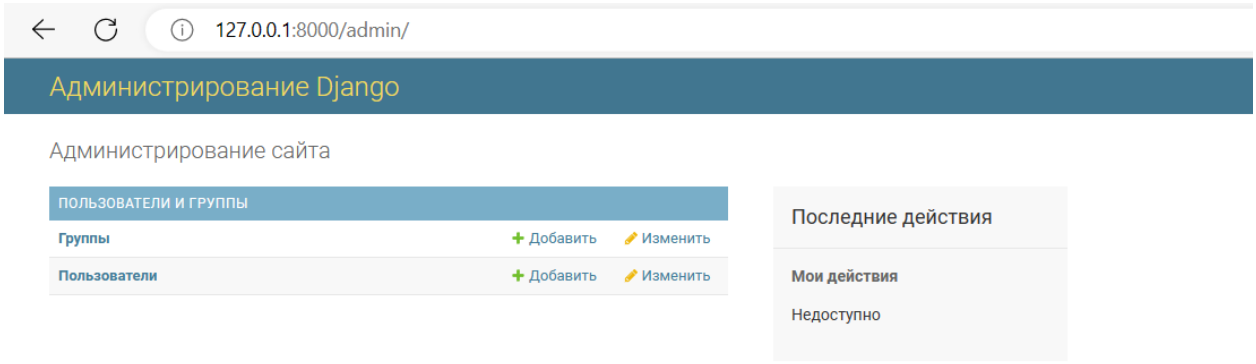


Рисунок 40

Пока конечно он пустой, поскольку мы не выполняли никаких действий, и они не были зарегистрированы в журнале.

Здесь можно будет просматривать действия общего характера, так и для отдельного приложения.

Но здесь мы не видим нашего нового приложения news, потому что мы его не зарегистрировали для админки.

Для этого вернемся в наше приложение news и откроем файл admin.py. И зарегистрируем наше приложение.

В файле импортируем приложение и напишем строку регистрации

```
from .models import News

admin.site.register(News)
```

Регистрация реализуется методом register() и передаем ему название нашего модели. После обновления мы увидим в списках наше приложение (рисунок 41)



Рисунок 41

Django автоматически добавляет к названию регистрируемого приложения -s. Поэтому получилось немного странно, что приложение с двумя s.

Если откроете эту вкладку, то увидите список наших новостей (рисунок 42)

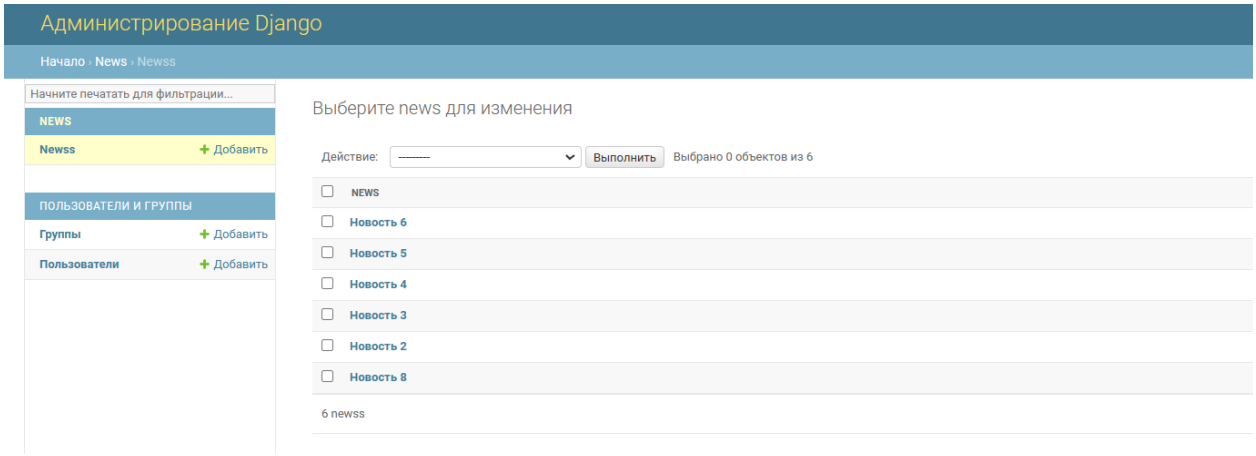


Рисунок 42

Здесь можно выбирать какие-то действия (рисунок 43).

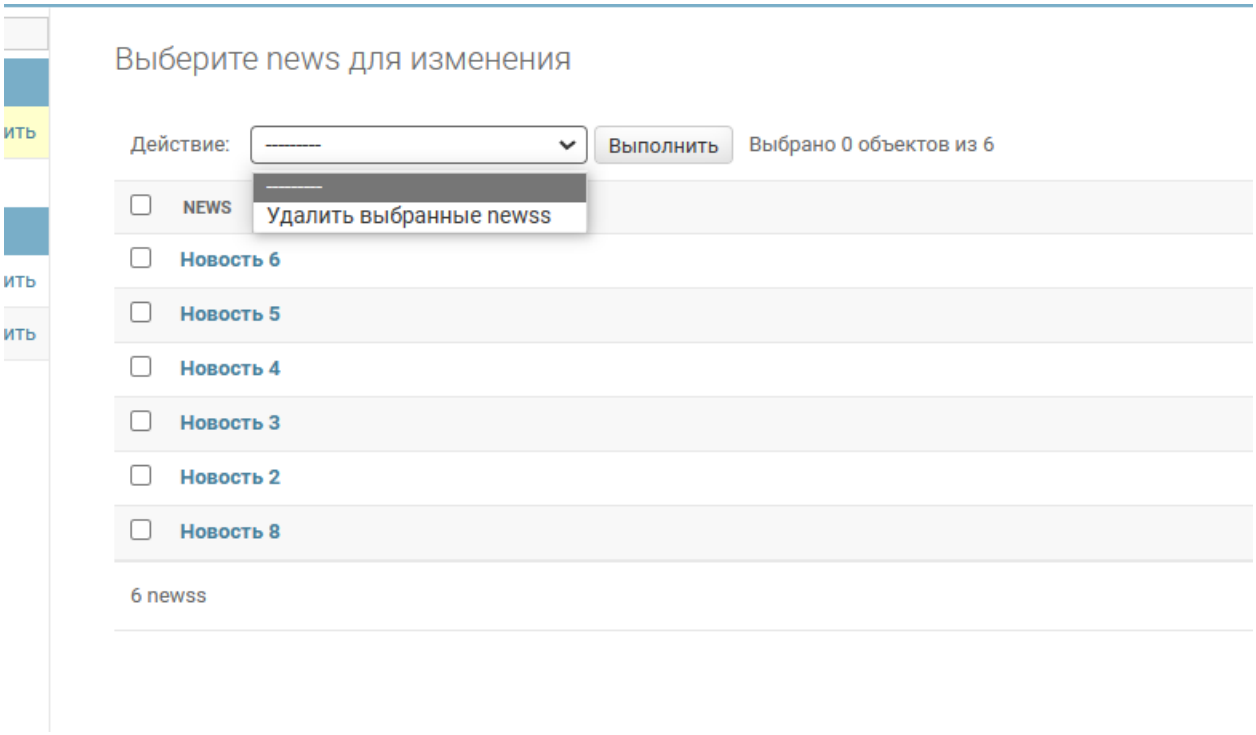


Рисунок 43

Можно добавить новую новость (рисунок 44)

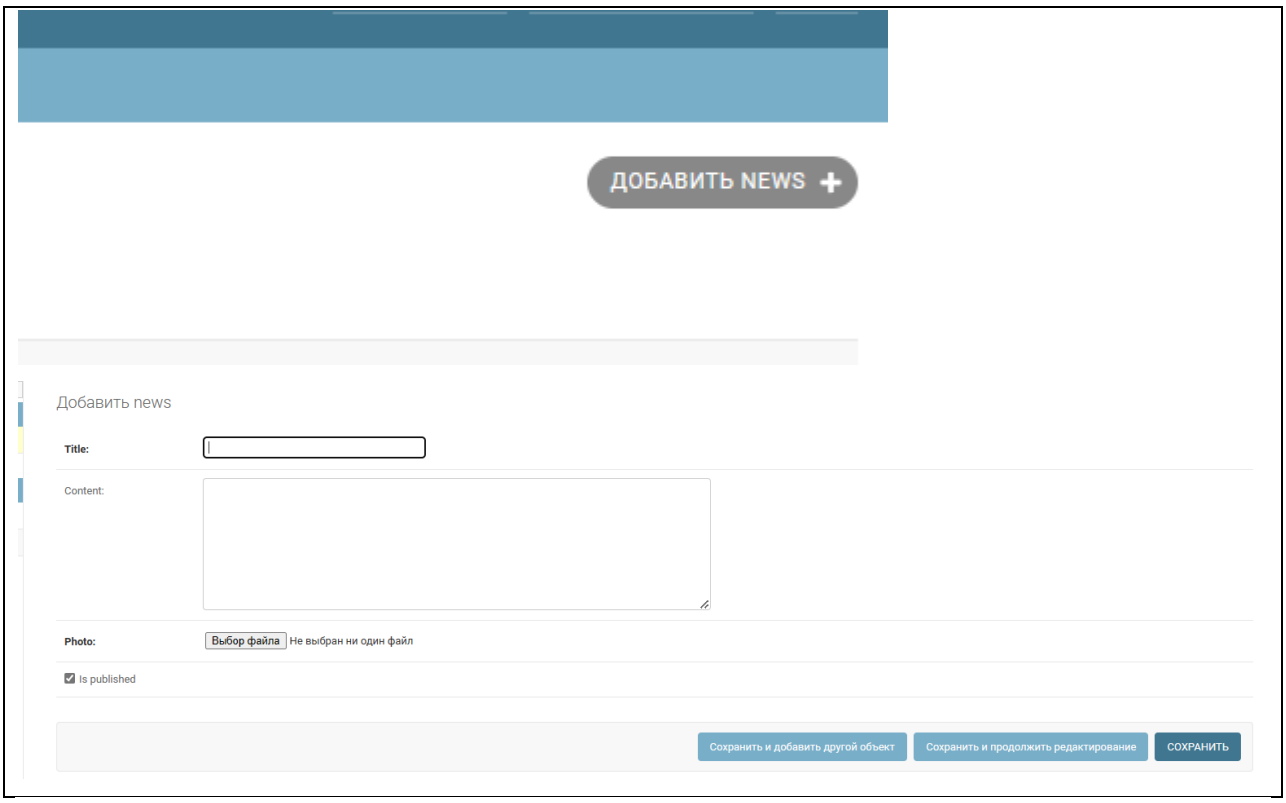


Рисунок 44

Можно отредактировать (рисунок 45)

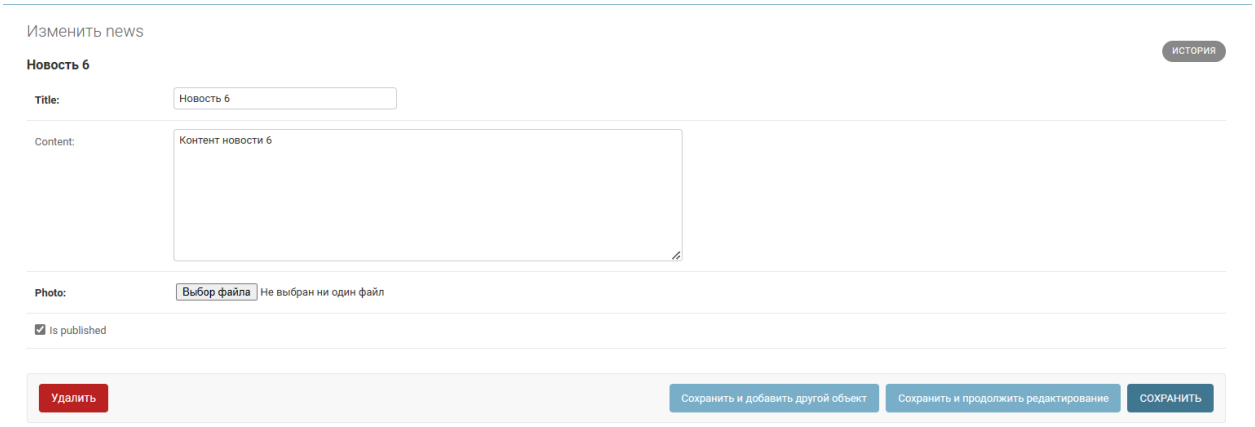


Рисунок 45

И конечно, провести настройки по свои пожелания. Например, назвать приложение более интересно, вывести другие поля, т.е., кроме title, вывести другие поля модели.

Все это делается в нашей модели. Т.е. переходим в файл models.py и создаем специальный подкласс Meta (рисунок 46)

```
    class Meta:
verbose_name='Новость'
```

Атрибут этого подкласса Meta `verbose_name` – это наименование модели в единственном числе.

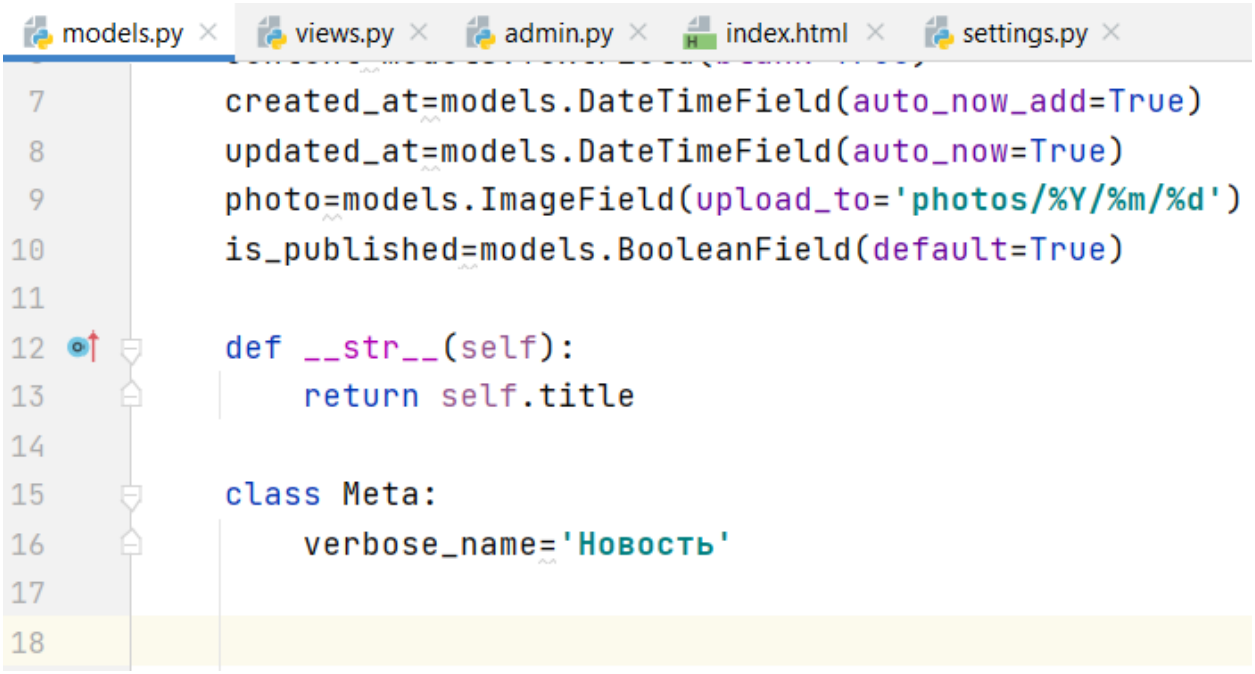


Рисунок 46

Давайте посмотрим результат в админке (рисунок 47).

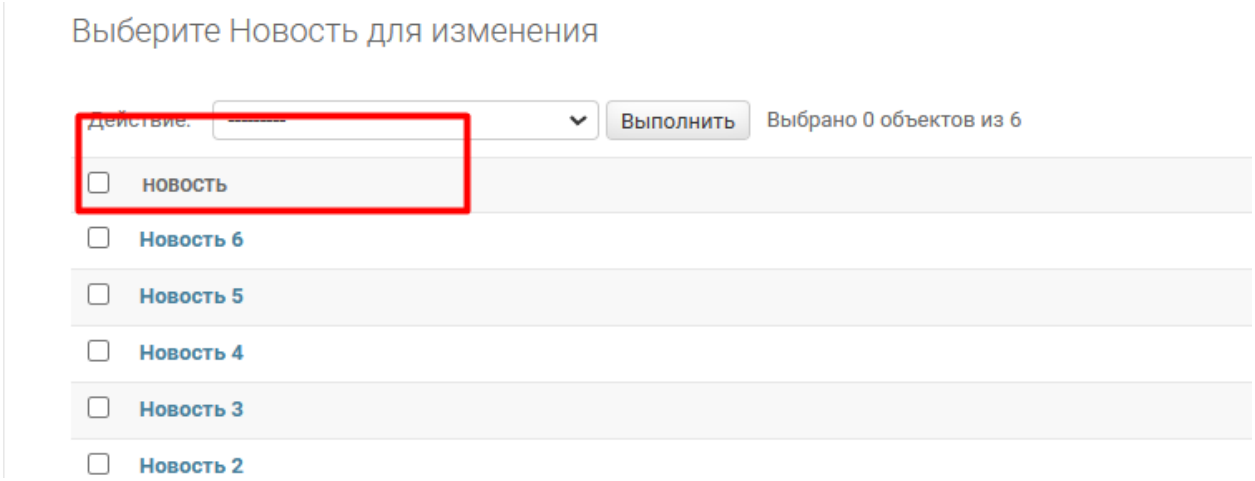


Рисунок 47

Перейдя на страницу изменения любой новости мы увидим обновлённый заголовок (рисунок 48).

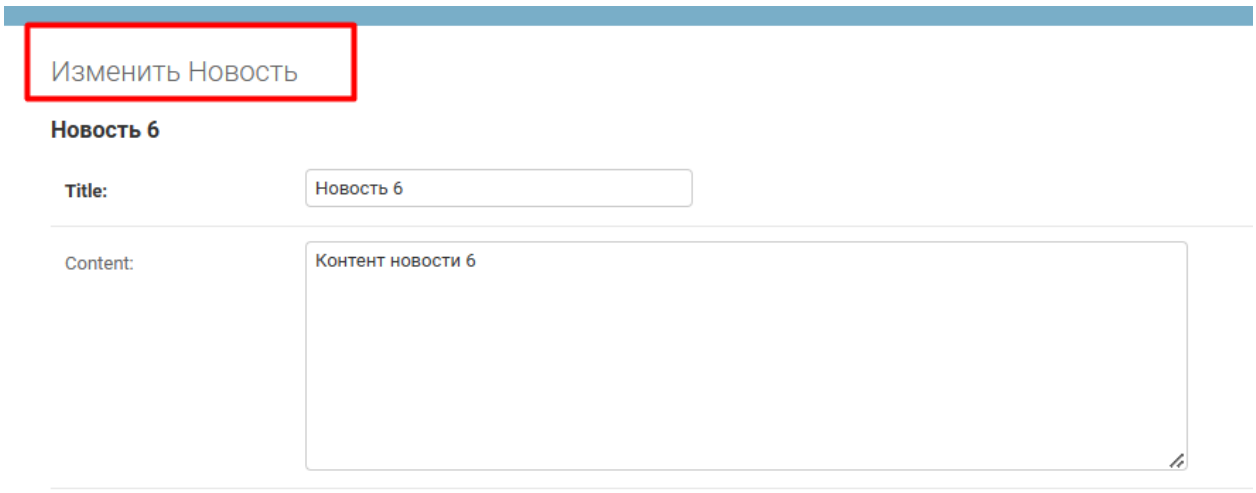


Рисунок 48

Следующий атрибут verbose_name_plural – это наименование модели во множественном числе (рисунок 49).

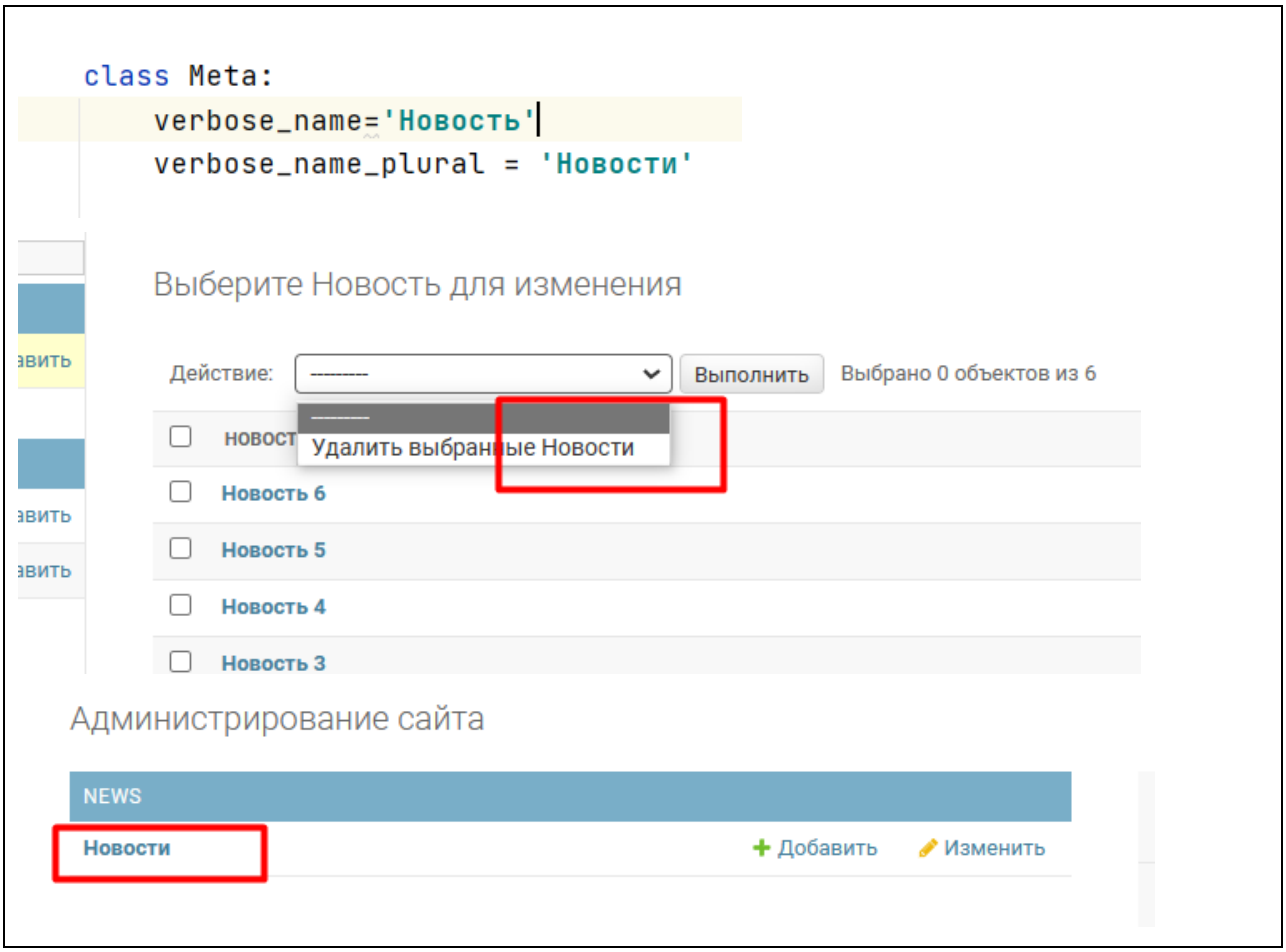


Рисунок 49

Можем поменять порядок следования новостей.

И это делается с помощью атрибута `ordering=['created_at',]`

Параметрами которого являются сортируемые поля (рисунок 50)



Рисунок 50

Этот порядок сортировки влияет не только на админку, но и на пользовательскую часть. И тогда во view.py можно не указывать порядок (рисунок 51)

```
def index(request):
    news = News.objects.order_by('-created_at')
    context={

def index(request):
    news = News.objects.all()
    context={
        'news': news,
        'title': 'Список новостей'
    }
```

Рисунок 51

И тогда порядок сортировки будет взят из модели.

Обновим страницу клиента (рисунок 52)



Рисунок 52

```
class Meta:
    verbose_name='Новость'
    verbose_name_plural = 'Новости'
    ordering=['-created_at',]
```

Рисунок 53

Список новостей

Новость 6

Контент новости 6

2023-03-17 16:45:50

Новость 5

Контент новости 5

2023-03-17 16:45:39

Новость 4

Контент новости 4

2023-03-17 16:31:56

Новость 3

Контент новости 3

2023-03-17 16:27:39

Новость 2

Контент новости 2

2023-03-17 16:25:08

Новость 8

Контент 8

2023-03-17 16:15:28

Рисунок 54

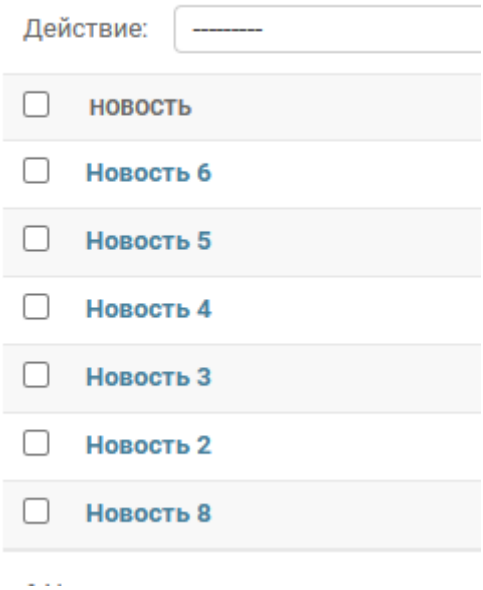


Рисунок 55

Поменяем еще слово NEWS на русский язык (рисунок 56)

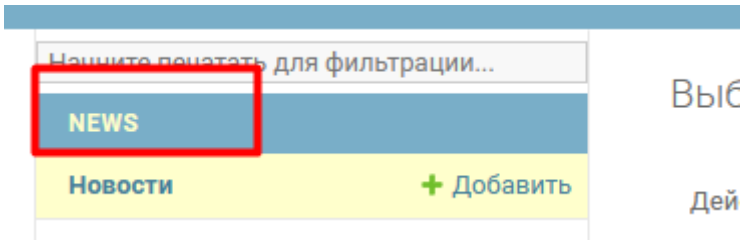


Рисунок 56

Здесь придёт на помощь конфигурация нашего приложения (рисунок 57, 58)

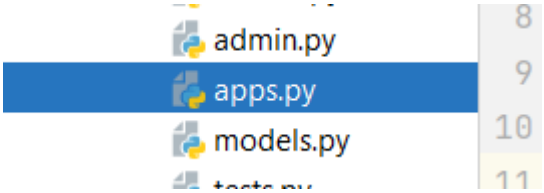


Рисунок 57

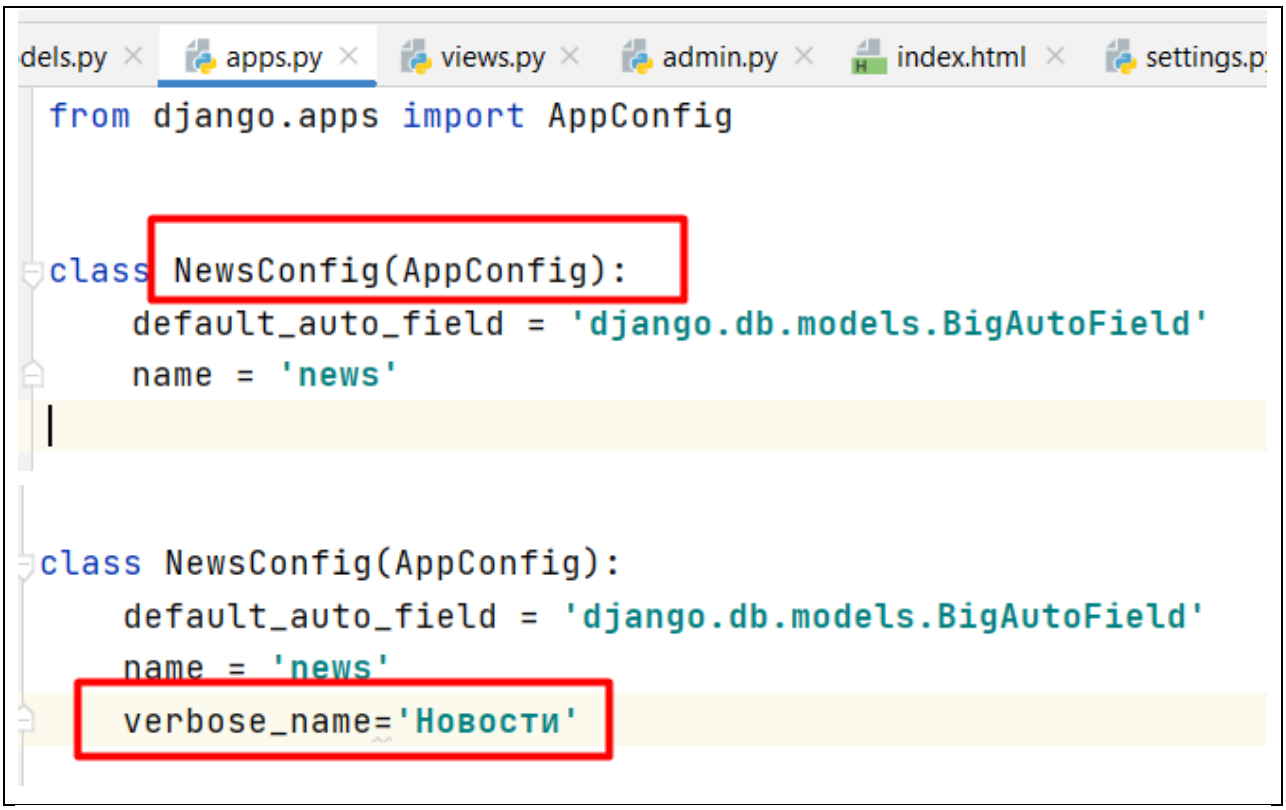


Рисунок 58

Обновим страницы браузера (рисунок 59)

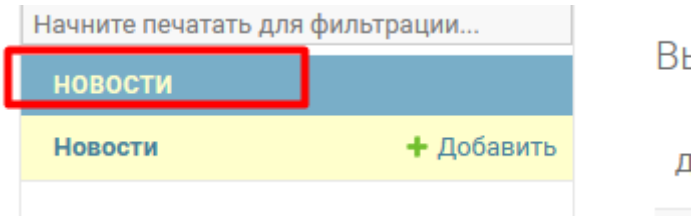


Рисунок 59

Следующим шагом добавим поля в списке новостей в админке. Это делается в admin.py с помощью класса редактора, в котором можно настроить представления модели в админке (рисунок 60).

```
class NewsAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'created_at', 'updated_at','is_published')
```

Рисунок 60

Для того чтобы это вступило в действие нам необходимо его зарегистрировать вместе с News (рисунок 61).

```python
class NewsAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'created_at', 'updated_at','is_published')


admin.site.register(News, NewsAdmin)
```

Рисунок 61

Необходимо обратить внимание, что здесь важен порядок. Т.е. вначале регистрируем саму модель News, а потом ее настраиваем.

NewsAdmin- это настраивающий класс и конечно, логично вначале создать класс, а потом его настраивать.

Обновим страницу браузера админки (рисунок 62).



Рисунок 62

Как мы видим поля названы на английском, как в модели.

Эту ситуацию можно исправить, если передать в конструкторы, соответствующие аргументы (рисунок 63, 64).

```python
class News(models.Model):
    title=models.CharField(max_length=150, verbose_name='Наименование')
    content=models.TextField(blank=True)
```

Рисунок 63

```python
class News(models.Model):
    title=models.CharField(max_length=150, verbose_name='Наименование')
    content=models.TextField(blank=True, verbose_name='Контент')
    created_at=models.DateTimeField(auto_now_add=True, verbose_name='Дата публикации')
    updated_at=models.DateTimeField(auto_now=True, verbose_name='Обновлено')
    photo=models.ImageField(upload_to='photos/%Y/%m/%d', verbose_name='Фото')
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
```

Рисунок 64

Обновим страницу браузера (рисунок 64)

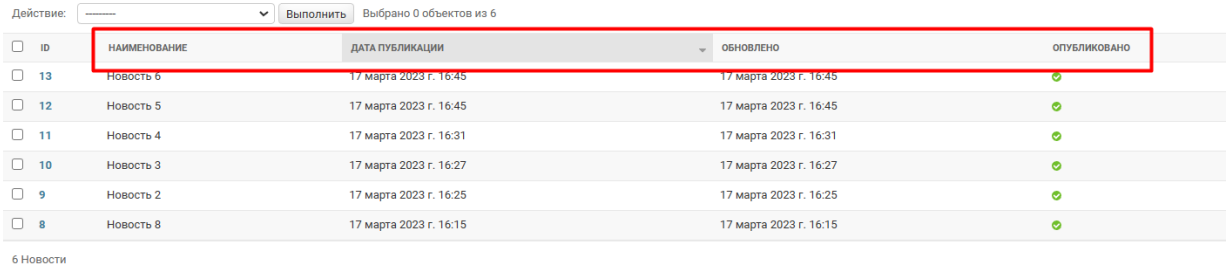


Рисунок 65

Для редактирования нам доступен id (рисунок 66)

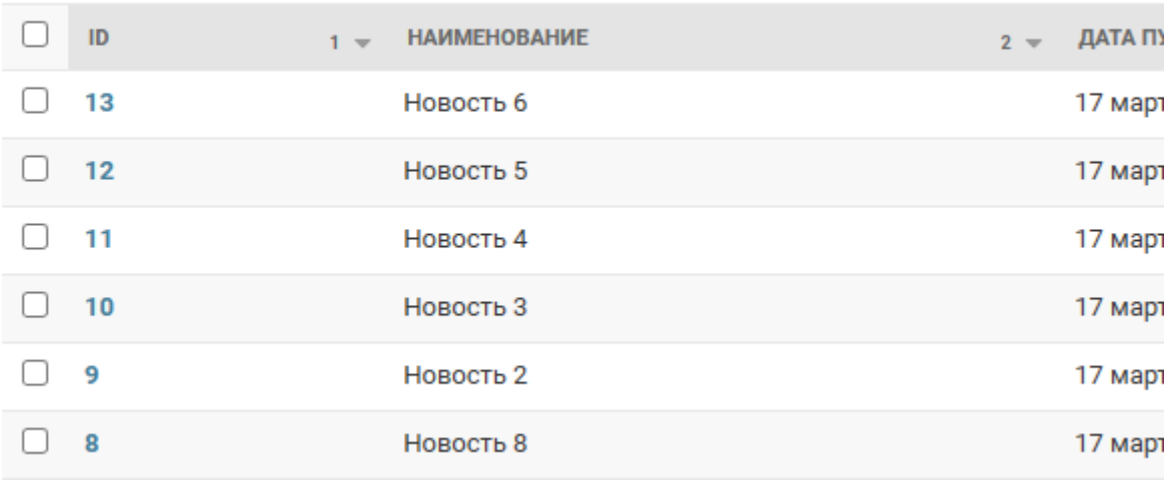


Рисунок 66

Если, необходимо сделать, чтобы ссылками были другие записи поля для их редактирования, то для этого необходимо обратиться к атрибуту (рисунок 67, 68) `list_display_links`

```
class NewsAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'created_at', 'updated_at','is_published')
    list_display_links =('id','title')
```

Рисунок 67

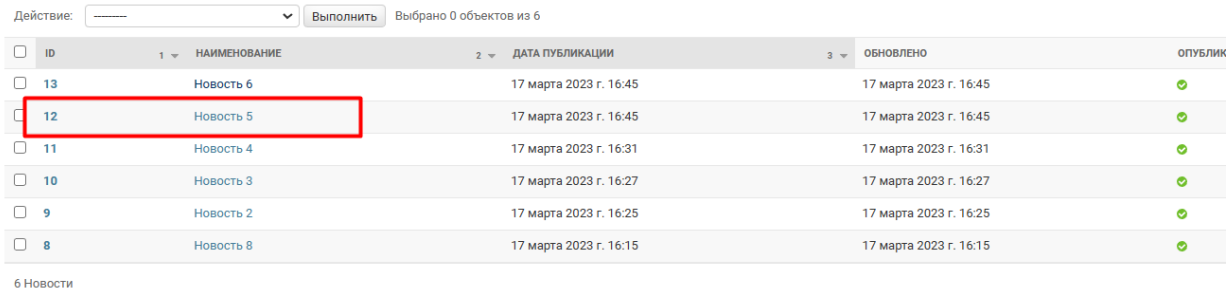


Рисунок 68

Настроим поля по которым можно проводить поиск. Для этого используется атрибут search_fields (рисунок 69)

```
list_display = ( 'id' ,  'title' ,  'create
list_display_links =('id','title')
search_fields =('title','content')
```

Рисунок 69

После обновления появится поле поиска (рисунок 70)

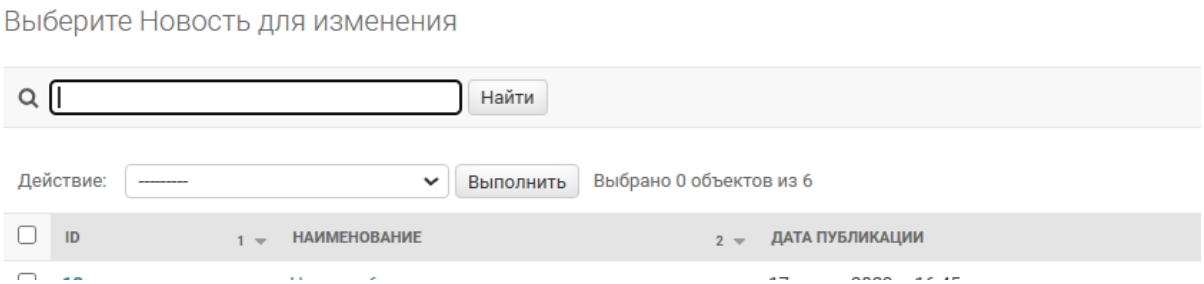


Рисунок 70

Давайте попробуем добавить новость из админки (рисунок 71).

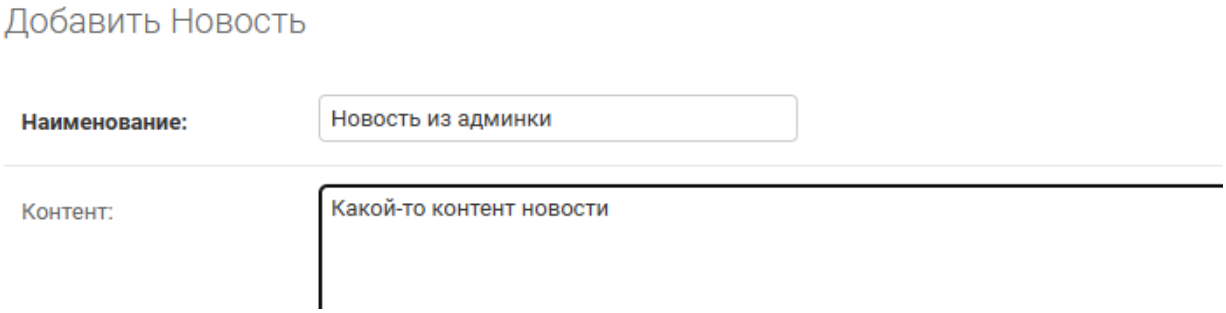


Рисунок 71

Если мы попробуем Сохранить, то Django выведет ошибку, потому что не было заполнено обязательное поле фотографии (рисунок 72).

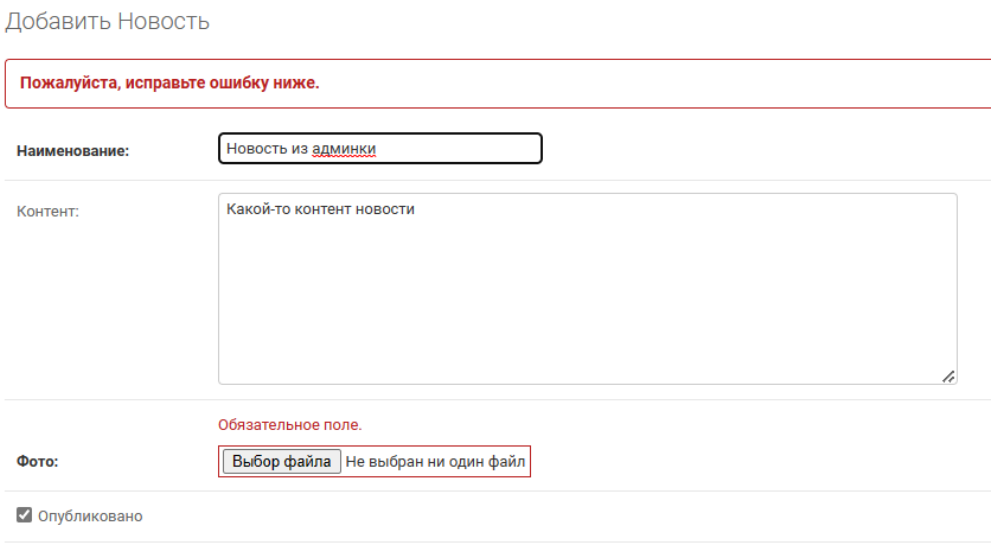


Рисунок 72

Добавим фотографию (рисунок 73)

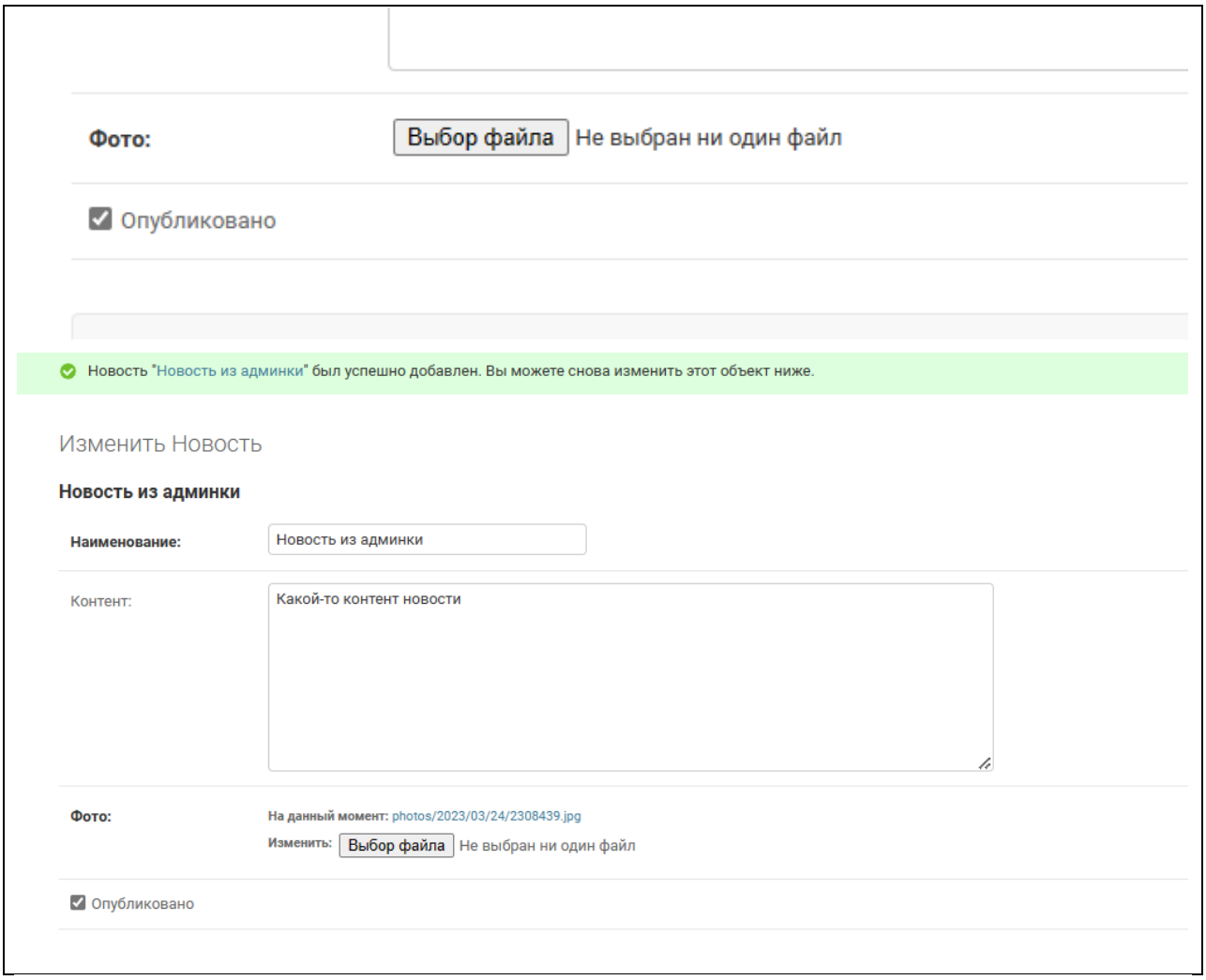


Рисунок 73

При этом в самом проекте появилась новая папка media (рисунок 74)

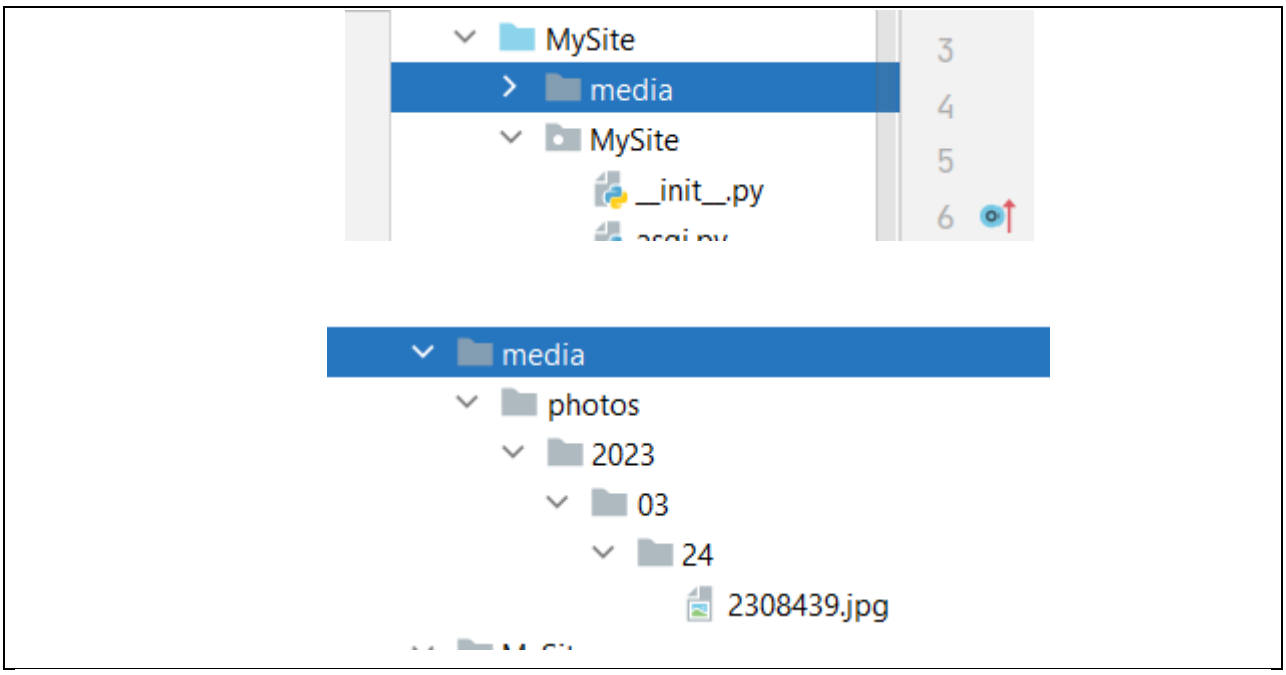


Рисунок 74

Если мы попробуем создавать новые записи, то появятся ошибки, связанные с заполнением обязательных полей.

Здесь необходимо сказать о валидации модели. Исправим это установление атрибута полю Фото как не обязательное для заполнения.

Для этого в файле модели в описании атрибутов поля Фото добавим атрибут blank=True (рисунок 75)

```
updated_at=models.DateTimeField(auto_now=True, verbose_name='Обновлено')
photo=models.ImageField(upload_to='photos/%Y/%m/%d', verbose_name='Фото', blank=True)
is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
```

Рисунок 75

Но чтобы внести все эти изменения в модели, необходимо создать файл миграции с помощью уже известной команды (рисунок 76)

python manage.py makemigrations

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py makemigrations
Migrations for 'news':
  news\migrations\0002_alter_news_options_alter_news_content_and_more.py
    - Change Meta options on news
    - Alter field content on news
    - Alter field created_at on news
    - Alter field is_published on news
    - Alter field photo on news
    - Alter field title on news
    - Alter field updated_at on news

(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>
```

Рисунок 76

Как видим из рисунка, был выведен список всех изменений модели и в дереве списка создан новый файл миграции (рисунок 77)

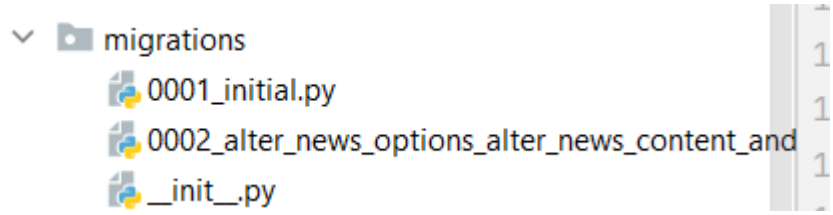


Рисунок 78

Далее выполняем саму миграцию с помощью тоже уже известной команды (рисунок 78)

python manage.py migrate

```
    - Alter field updated_at on news
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, news, sessions
Running migrations:
  Applying news.0002_alter_news_options_alter_news_content_and_more... OK

(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>
```

Рисунок 79

Теперь попробуем изменить запись без прикрепления фото.